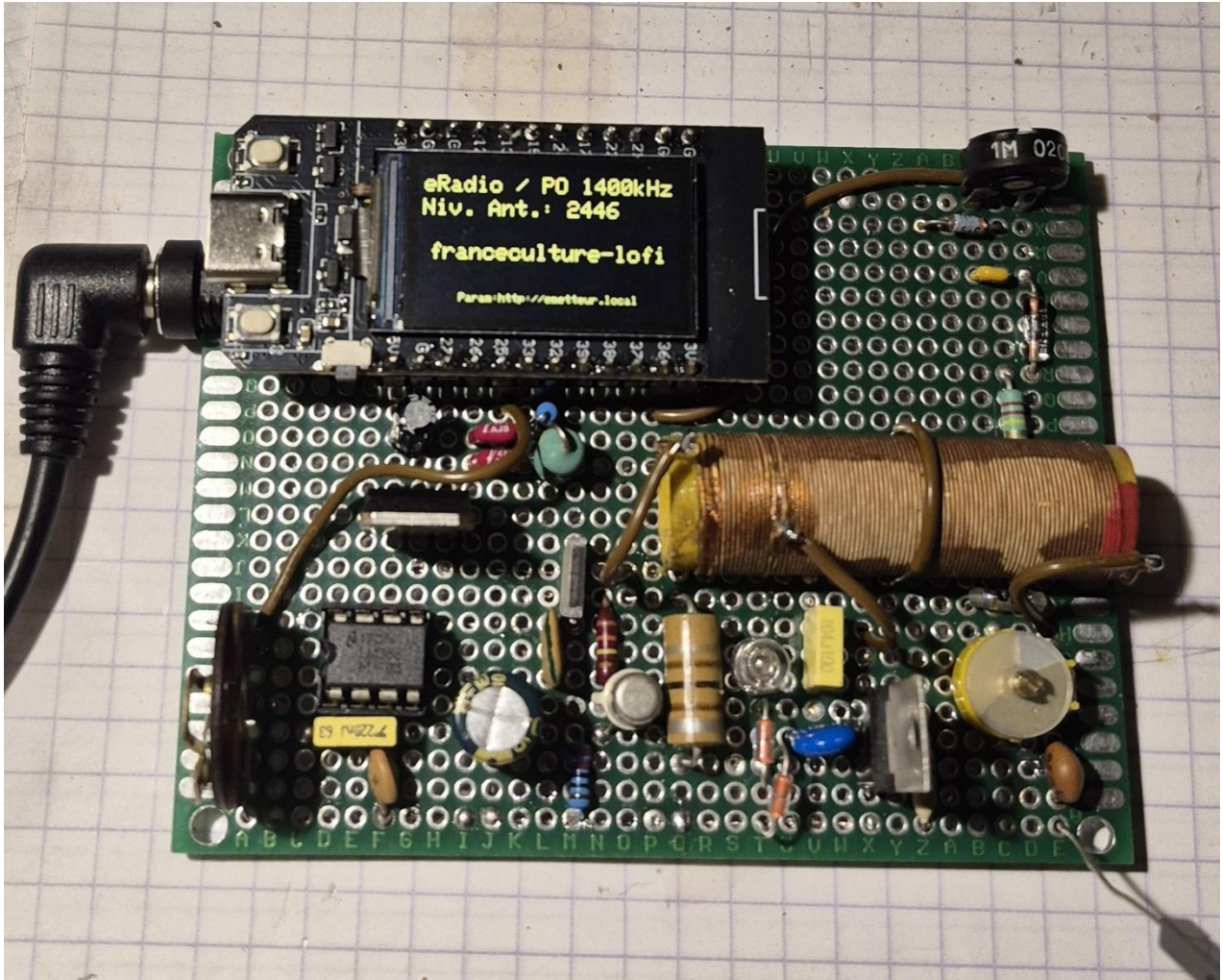


Building a small C-class AM-MW transmitter



Author : Philippe de Craene dcphilippe@yahoo.fr

January 2026

Manual version : 1.0

Program version : 2.1

Before the world of the internet and even the FM radio stations, the common way to be connected to the World was the radio diffusion. It was in AM, for Amplitude Modulation, and this ancestral way to communicate was very popular from the years 1920 to 2000. In those times radio set used to take a great place in a house, and most of times cost several months of a labor salary.

Quite suddenly all those radio stations disappeared in the early years 2000. For instance, there is no more radio stations in French language in Europe, the only one in the LW range is the BBC in English.

I collect radios, more or less old, installed quite everywhere in my house, and I'm suffering not to be able to listen them anymore.

There are plenty of diagrams on how to build a low power transmitter with 2 or 3 transistors, generally powerless. Also, there are ready to use Chinese kits, but...I decide to build my own AM transmitters, and I'm explaining here a way to build a very simple one, and a little more complex transmitter that offers broadcasting all over a house, that means around 10 meters.

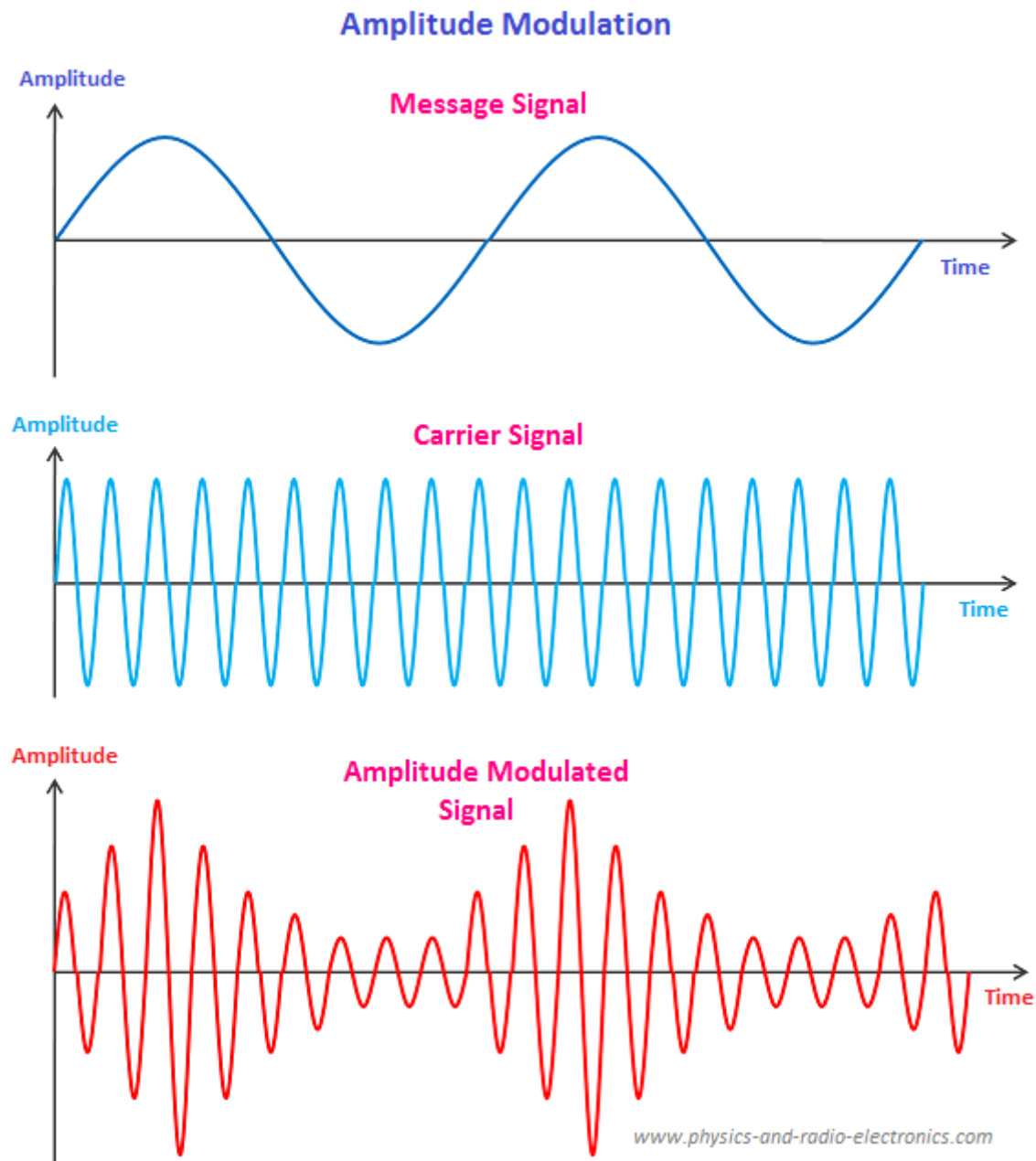
Table of content

Table of content.....	2
Introduction.....	3
Very first test with a standalone ESP32.....	4
Description	4
Diagram of the improved transmitter with ESP32 + 2N7000	5
Second test with a ESP32 with external modulation	6
Description	6
Diagram of the transmitter ESP32 with external modulation	6
ESP32 code version (V2) with external modulation feature	6
Improved MW-AM transmitter	16
Description: the choose of a C-class amplifier	16
The diagram of the ESP32 with a C-class transmitter	17
List of materials	18

Introduction

There are many literatures that explain what is AM transmission. An old AM radio set is able to receive a carrier frequency, for instance 240kHz in the LW range or 1500kHz in the MW range, and if this carrier frequency has its amplitude modulated by a audio signal, the radio set will transmit this audio signal through a loudspeaker.

Here is a very good explanation of AM: <https://www.physics-and-radio-electronics.com/blog/amplitude-modulation>



There are several reasons why I want to build my own transmitter:

- 1- For most of what exist as transmitters, the audio input is either trivial just as an input, or the audio input consists of a Bluetooth. But my opinion is listening a radio set is NOT listening MP3 of the smartphone Bluetooth.
Moreover, these implementations require an additional device, whether wired or not.
My opinion is that a radio set is used to listen... a radio station.

The audio input could be a FM receiver added to the transmitter circuit....

And why not the simplest implementation around a single small module like an ESP32 as a web-radio receiver?

So the source will be built around an ESP32 with the very popular Audio.h library from Schreibfaul1 available here: <https://github.com/schreibfaul1/ESP32-audioI2S>

- 2- As now we are swimming in the digital world, why this ESP32 would not generate the carrier frequency? This solution offers the advantage of massaging oneself with an oscillator, oscillator that must be as stable as possible.

Very first test with a standalone ESP32

Description

I had a lot of fun imagining a mini radio transmitter reduced to the use alone of an MPU like an ESP32 (the basic one with a DAC)

I get a ESP TTGO T-Display and I create a code that :

- Receives a web-radio station, and outputs an audio signal
- Allows to manage web-station URL, to add or delete them, with its integrated webserver interface accessible from any (same) wifi smartphone or PC
- Allows the web-radio station changes with its integrated 2 push-buttons,
- Generates the carrier RF signal
- With the PWM properties of ledc, this carrier frequency can be modulated by the audio signal.

The result is the project explained here:

<https://www.hackster.io/philippedc/esp32-ttgo-webradio-lw-mw-am-broadcast-radio-transmitter-d87a59>

All files, diagrams and codes are also available here:

<https://github.com/philippedc/ESP32-TTGO-T-Display-webradio-LW-MW-AM-broadcasted/tree/main>

The device is well working as long as the loop antenna is very close to the old radio to properly operate.

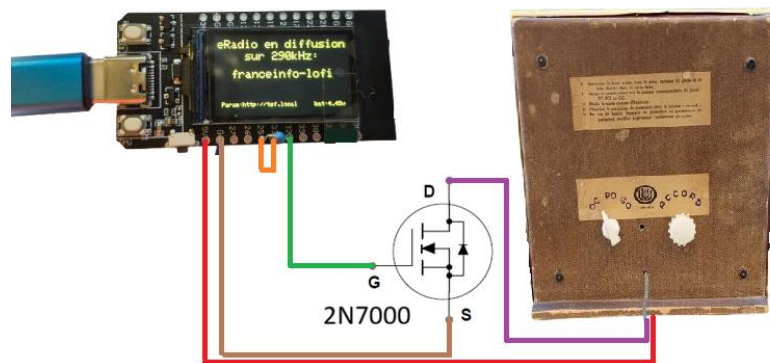
Thanks, the loop antenna has a very bad Q, otherwise the voltage would be higher than an ESP32 would tolerate...

A solution to add some power is to add a small MOSFET like the very low cost 2N7000 between the ESP32 output and the loop antenna input.

In this case the 2N7000 amplifies the ESP32 modulated signal and offers the possibility to use a higher power supply voltage. The power is considerably increase.

Diagram of the improved transmitter with ESP32 + 2N7000

webradio to LW/AM Broadcast transmitter



The ESP32 works with a 3.3V power supply, the increase of power is due by the higher voltage operation of the MOSFET. In my test experience it is 4V of a lipo battery, but it can be 9 or 12V for much more performances.

The other disadvantage of this very small transmitter is that the carrier frequency must be under 250kHz.

This is due to the ledc properties: the PWM resolution depends of the PWM frequency:

- PWM frequency above 60kHz => 10 bits resolution
- 125kHz - 250kHz => 8 bits
- 500kHz - 1 MHz => 6 bits
- 250kHz - 500kHz => 7 bits
- 60kHz - 125kHz => 9 bits

The carrier frequency must stay under 240kHz to keep a minimal audio quality. So it is a LW transmitter.

Please note that as the carrier is a square signal, it is still possible to listen broadcast on MW range, in this case the radio set is tuned to the harmonic 3, $3 * 240 = 720\text{kHz}$.

Another disadvantage is that the domestic wifi must be available where the transmitter (and the radio set) is placed.

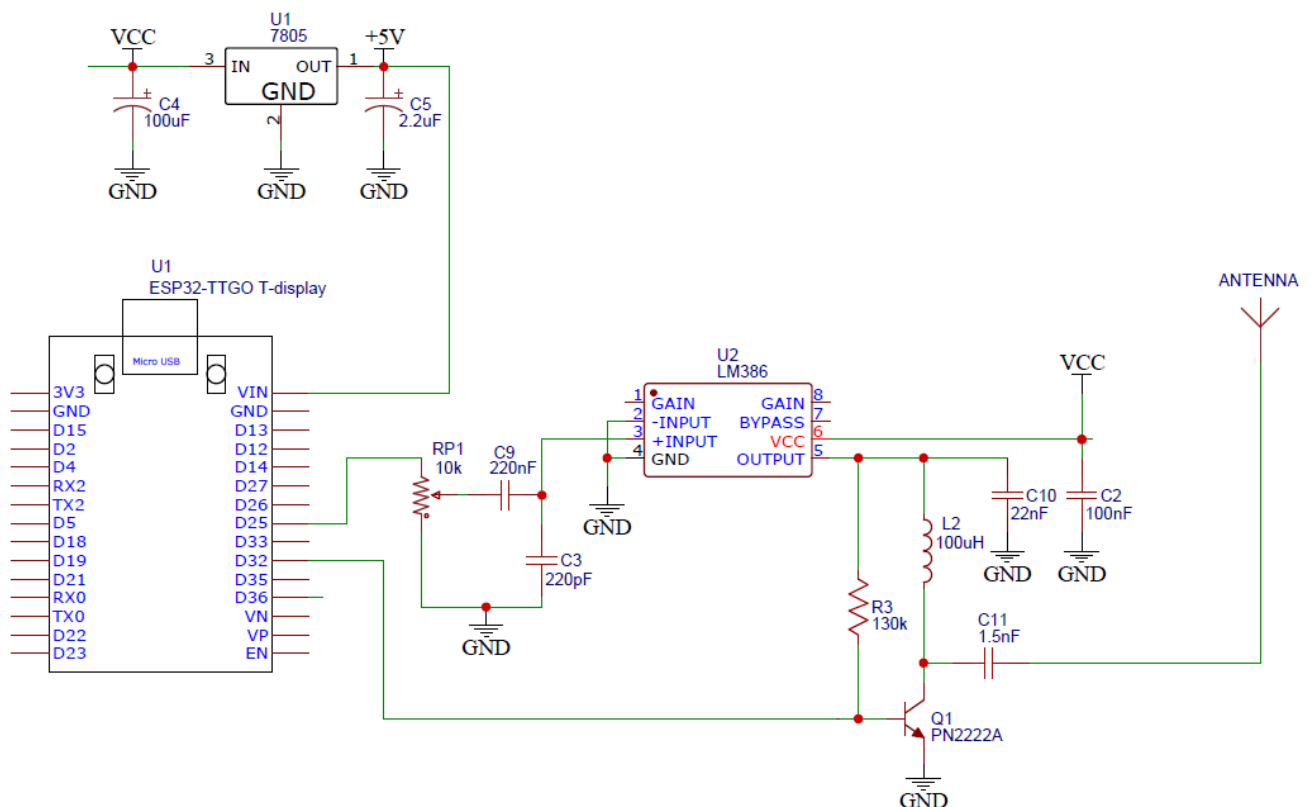
So, we will see how to build a transmitter in MW with a best broadcast ability.

Second test with a ESP32 with external modulation

Description

This time the ESP32 provides separately the audio and the carrier signals, the mix of them is made by external components.

Diagram of the transmitter ESP32 with external modulation



Q1 is the mixer transistor. The Base receives the 3.3V amplitude square carrier signal, on the Collector this signal is amplified. The power supply of Q1 will vary according to the audio signal.

As some amplification is needed, also some power because the output gives the current for Q1; I could use 2 transistors to do this task; I've chosen a simpler way with the use of a LM386.

ESP32 code version (V2) with external modulation feature

The code for the ESP32 is modified so there is no more PWM modulation. It is available here:

<https://github.com/philippedc/ESP32-TTGO-T-Display-webradio-LW-MW-AM-broadcasted>

IMPORTANT note:

The code is working for Arduino IDE version 1.8.16. It will not work with version 2.x.x

The espressif ESP32 version must be 2.0.3 for the Audio.h 2.0.6

These are quite old versions, but each new version carries many changes in features and incompatibilities. The advantage of these old versions is that this code is working well.

```

/*=====
test for webradio receiver AM broadcast transmitter

-> ESP32 TTGO t-display ST7789V SPI bus TFT
    default code https://github.com/Xinyuan-LilyGO/TTGO-T-
Display/blob/master/TFT_eSPI/examples/FactoryTest/FactoryTest.ino

-> V2 gives separate output for audio (webradio) and carrier RF. That allows:
    - to transmit in MW range frequencies like 1500kHz, the highest RF frequency is, the best
    propagation is, for a same lenght of antenna
    - to consideraly increase power transmission with a power supply larger than 3.3V

IDE version 1.8.16 with
Expressif Sytems (ESP32) version 2.0.3
=> Audio.h v2.0.6

//-----
// Board      : "ESP32 Dev Module"
//-----
// Upload Speed : "921600"
// CPU Frequency : "240MHz (WiFi/BT)"
// Flash Mode    : "QIO"
// Partition Scheme : "Huge APP (3MB APP no OTA/1MB SPIFFS)"
// PSRAM        : "Disabled"

ESP32-TTGO_T_Display pinup
-----

GPIO25  => DAC1 output webradio output
GPIO32  => RF output
GPIO36  => frLevel input

GPIO35  => push-button station +
GPIO00  => push-button station -

//=====

```

author : Philippe de Craene <dcphilippe@yahoo.fr>

```

version history
-----
v0      24 oct 25   - first test webradio to AM transmitter
v0.2    26 oct 25   - add mDNS and battery voltage measure
v0.3    26 oct 25   - give the choice of the rf frequency
v2.0    10 jan 26   - separate audio and RF output

*/

// parameters
//-----
const String VERSION = "v2.0";          // code version
unsigned int rfFreq = 1500;              // carrier frequency in kHz
const char* dnshost = "emetteur";       // DNS name

byte volume = 21;                       // initial audio volume
int radioPlayingDelay = 120;             // radio autostop in minutes
int maxTitleLenght = 40;                 // max lenght of radio title

// the RF modulation
//-----
#define rfOut 32
#define PWM_channel 0 // pwm channel - choise = [0 - 15]
#define PWM_Res 2 // [0 - 3] 2 bits resolution - choise = [1 - 16]
/* resolution depends of the PWM frequency:
 * 500kHz - 1 MHz => 6 bits
 * 250kHz - 500kHz => 7 bits
 * 125kHz - 250kHz => 8 bits
 * 60kHz - 125kHz => 9 bits
 * above 60kHz => 10 bits
 */
int duty = 2;
#define rfLevelPin 36

```



```

// the push-buttons
//-----
#define pb1      35
#define pb2      0

// wifi & web server
//-----
#include <WiFi.h>
#include <AsyncTCP.h>           // required for ESPAsyncWebServer.h
#include <ESPAsyncWebServer.h> // https://github.com/me-no-dev/ESPAsyncWebServer
#include <ESPAsyncWiFiManager.h> // https://github.com/alanswx/ESPAsyncWiFiManager
// web server                   // exemple: https://electropeak.com/learn/create-a-web-server-w-esp32/
AsyncWebServer server(80);     // set web server on port #80
DNSServer dns;
#include <ESPmDNS.h>           // https://techtutorialsx.com/2022/01/24/esp32-webserial-and-mdns/
mDNS/

// display & graphics
//-----
//##### DON'T FORGET TO UPDATE THE User_Setup.h FILE IN THE LIBRARY #####
//##### SETUP FILE NAMED: User_Setup_Select.h TO REFLECT THE TTGO DISPLAY !!!
// TFT Pins has been set in the TFT_eSPI library in the User Setup file TTGO_T_Display.h
// #define TFT_MOSI      19
// #define TFT_SCLK      18
// #define TFT_CS        5
// #define TFT_DC        16
// #define TFT_RST       23
// #define TFT_BL        4           // Display backlight control pin
#include "TFT_eSPI.h"             // https://github.com/Bodmer/TFT_eSPI
TFT_eSPI tft = TFT_eSPI();      // Use hardware SPI

#define backgroundColor  TFT_BLACK
#define textColor        TFT_GOLD // TFT_WHITE
byte screenRotation = 1;        // 1 or 3

// Audio
//-----
#include "Audio.h"               // https://github.com/schreibfaul1/ESP32-audioI2S
Audio audio(true, I2S_DAC_CHANNEL_RIGHT_EN); // only DAC1 on GPIO25

// to keep data
//-----
#include "SPIFFS.h"             // keeps stations.txt file

// EEPROM
#include <Preferences.h> // https://randomnerdtutorials.com/esp32-save-data-permanently-preferences/
Preferences pref;

// radio station list
//-----
String stations[] = {
  "live-reflector.ice.infomaniak.ch/rfiafrique-64.mp3",
  "live02.rfi.fr/rfiafrique-64.mp3",
  "icecast.radiofrance.fr/franceculture-lofi.mp3",
  "icecast.radiofrance.fr/franceinter-lofi.mp3",
  "icecast.radiofrance.fr/franceinfo-lofi.mp3",
  "icecast.radiofrance.fr/francemusique-lofi.mp3",
  "stream.srg-ssr.ch/m/rsc_de/mp3_128",
  "ais-sa2.cdnstream1.com/2208_128.mp3",
  "radioseribatu.out.airtime.pro:8000/radioseribatu_a",
  "centovall.instainternet.com:8050/stream",
  " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ",
  " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ",
  " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ",
  " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " ", " "
};
byte radioList = 10;           // total number of stations
int radioNum = 0;              // current radio station index number
String radioName = "", memoRadioName = "";
String radioTitle = "", memoRadioTitle = "";

// other global variables
//-----
String ipAddress = "http://";
String dnsName = "http://";
bool change = true;
int radioCounter = radioPlayingDelay;

```



```

// Define the html pages
//-----
// example: https://esp32io.com/tutorials/esp32-web-server-multiple-pages

const char view_html[] PROGMEM = R"=====(
<!DOCTYPE html><html lang=fr-FR><head><title>Emetteur Radio chez Fifi</title>
<head><meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" href="data:,">

<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}
.button { background-color: #8A0808; border: none; color: white; padding: 16px 40px;
text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}
.button2 {background-color: #32CD99;} .button3 {background-color: #08298A;}
</style></head>

<BODY><h1>Emetteur Radio chez Fifi</h1>
%RADIOPLACEHOLDER%
</body></html>
)=====";

const char confirmdelete_html[] PROGMEM = R"=====(
<!DOCTYPE html><html lang=fr-FR><head><title>HEmetteur Radio chez Fifi</title>
<head><meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" href="data:,">

<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}
.button { background-color: #8A0808; border: none; color: white; padding: 16px 40px;
text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}
.button2 {background-color: #32CD99;} .button3 {background-color: #08298A;}
</style></head>

<BODY><h1>Emetteur Radio chez Fifi</h1>
%CONFIRMDELETEPLACEHOLDER%
</body></html>
)=====";

//
// SETUP
//-----

void setup() {

// Assign loop2() to core 0
//-----
xTaskCreatePinnedToCore(
  loop2,          // Function to implement the task
  "loop2",        // Name of the task
  50000,          // Stack size in bytes
  NULL,           // Task input parameter
  5,              // Priority of the task
  NULL,           // Task handle.
  0               // Core where the task should run
);

// start the console
//-----
Serial.begin(115200);          // only here to economize memory
while(!Serial);               // wait for user to open terminal
Serial.println(F("Starting..."));

// start SPIFFS
//-----
// info: https://www.programmingelectronics.com/spiffs-esp32/
#ifdef INIT_SPIFFS
  Serial.println(F("Formating SPIFFS then copy SDcard files to it"));
  if(SPIFFS.format()) Serial.println(F("SPIFFS erased successfully.));
#endif
  SPIFFS.begin(true);

// get the radio stations list
  ListDir(SPIFFS, "/", 0);
  File f = SPIFFS.open("/stations.txt"); // ouverture du fichier en lecture
  if( !f ) {
    Serial.println(F("No file /station.txt, default radio list is used"));
    updateRadioList();
  }
  else if( f.size() > 0 ) {
    Serial.println(F("web radio station list:"));
    radioList = 0;
    char c;
    stations[0] = "";
    for( unsigned int i=0; i< f.size(); i++ ) {

```

```

        c = f.read();
        if( c != '\r' ) {
            if( c != '\n' ) stations[radiolist] += c;
        }
        else {
            Serial.printf("n%d = ", radiolist +1); Serial.println(stations[radiolist]);
            radiolist++;
            stations[radiolist] = "";
        }
    } // end of for
    f.close();
} // end of else f.size() > 0
else {
    Serial.println(F("File /station.TXT is empty, default radio list is used"));
    f.close();
    UpdateRadioList();
}
Serial.printf("total of web radio stations = %d\n", radiolist);

// get parameters
//-----
pref.begin("parameters", false);
radioNum = pref.getUChar("radio", 0);
rfFreq = pref.getUInt("freq", rfFreq);
volume = pref.getUChar("vol", volume);
screenRotation = pref.getUChar("sr", screenRotation);

// start the display
//-----
tft.init();
tft.fillScreen(backgroundColor);
/*
Normally strings are printed relative to the top left corner but this can be
changed with the setTextDatum() function. The library has #defines for:

TL_DATUM = 0 = Top left
TC_DATUM = 1 = Top centre
TR_DATUM = 2 = Top right
ML_DATUM = 3 = Middle left
MC_DATUM = 4 = Middle centre
MR_DATUM = 5 = Middle right
BL_DATUM = 6 = Bottom left
BC_DATUM = 7 = Bottom centre
BR_DATUM = 8 = Bottom right

L_BASELINE = 9 = Left character baseline (Line the 'A' character would sit on)
C_BASELINE = 10 = Centre character baseline
R_BASELINE = 11 = Right character baseline
*/
tft.setRotation(screenRotation);
tft.setTextSize(2);
tft.setTextColors(textColor);
tft.setTextDatum(MC_DATUM);
tft.drawString("Pour se connecter", tft.width()/2, tft.height()/2 -40);
tft.drawString("au wifi:", tft.width()/2, tft.height()/2 -20);
tft.drawString("rechercher wifiAP", tft.width()/2, tft.height()/2 );
tft.drawString("puis aller sur :", tft.width()/2, tft.height()/2 +20);
tft.drawString("http://192.168.4.1", tft.width()/2, tft.height()/2 +40);

// WiFi Connect
//-----
AsyncWiFiManager wifiManager(&server,&dns);
//wifiManager.resetSettings(); // to reset saved settings
//wifiManager.setTimeout(240); // sets timeout until configuration portal gets turned off
if(!wifiManager.autoConnect("wifiAP")) {
    Serial.println(F("failed to connect and hit timeout: restart device"));
    delay(3000);
    ESP.restart(); // reset and try again, or maybe put it to deep sleep
    delay(5000);
}

// Connect to wi-Fi network with SSID and password
wifi.setSleep(false); // to test as sometime the radio stops
tft.fillScreen(backgroundColor);
tft.drawString("Emeteur Radio GO", tft.width()/2, tft.height()/2 -50);
tft.drawString(VERSION, tft.width()/2, tft.height()/2 -30);
tft.drawString("adresse wifi:", tft.width()/2, tft.height()/2 );
IPAddress += WiFi.localIP().toString();
tft.drawString(IPAddress, tft.width()/2, tft.height()/2 +20);

// initialize mDNS
if(!MDNS.begin(dnshost)) Serial.println(F("Error setting up MDNS responder!"));
else {
    dnsName += String(dnshost) + ".local";
}

```

```

    tft.drawString(dnsName,                tft.width()/2, tft.height()/2 +40);
}

// prepare the webserver requests
//-----
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send_P(200, "text/html", view_html, processor);
});
server.on("/stationsuiv", HTTP_GET, [](AsyncWebServerRequest *request) {
    ChangeRadio(HIGH);
    request->redirect("/");
});
server.on("/stationprec", HTTP_GET, [](AsyncWebServerRequest *request) {
    ChangeRadio(LOW);
    request->redirect("/");
});
server.on("/confirmdelete", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send_P(200, "text/html", confirmdelete_html, processor);
});
server.on("/deleteradio", HTTP_GET, [](AsyncWebServerRequest *request) {
    if( radioList > 1 ) {
        radioList--;
        for(byte i= radioNum; i< radioList; i++ ) stations[i] = stations[i+1];
        radioNum--;
        UpdateRadioList();
    }
    request->redirect("/");
});
server.on("/addradio", HTTP_GET, [](AsyncWebServerRequest *request) {
    if( request->hasParam("r")) {
        String newRadio = request->getParam("r")->value();
        if( newRadio.length() > 5 ) {
            radioNum = radioList;
            stations[radioNum] = newRadio;
            Serial.printf("New web radiostation %d: ", radioNum+1); Serial.println(newRadio);
            radioList++;
            UpdateRadioList();
            ListenRadio();
        }
    }
    request->redirect("/");
});
server.on("/freqplus", HTTP_GET, [](AsyncWebServerRequest *request) {
    rfFreq += 100;
    if( rfFreq > 1800 ) rfFreq = 1800;
    pref.putUInt("freq", rfFreq);
    RfTransmitter();
    change = true;
    request->redirect("/");
});
server.on("/freqmoins", HTTP_GET, [](AsyncWebServerRequest *request) {
    rfFreq -= 100;
    if( rfFreq < 1000 ) rfFreq = 1000;
    pref.putUInt("freq", rfFreq);
    RfTransmitter();
    change = true;
    request->redirect("/");
});
server.on("/modulplus", HTTP_GET, [](AsyncWebServerRequest *request) {
    if( ++volume > 21 ) volume = 21;
    pref.putUChar("vol", volume);
    request->redirect("/");
});
server.on("/modulmoins", HTTP_GET, [](AsyncWebServerRequest *request) {
    if( --volume < 12 ) volume = 12;
    pref.putUChar("vol", volume);
    request->redirect("/");
});
server.on("/rotation", HTTP_GET, [](AsyncWebServerRequest *request) {
    if( screenRotation == 3 ) screenRotation = 1;
    else screenRotation = 3;
    change = true;
    pref.putUChar("sr", screenRotation);
    request->redirect("/");
});
server.begin();

// set RF transmitter
//pinMode(rfOut, OUTPUT_OPEN_DRAIN);
RfTransmitter();

// then to end setup
//-----
pinMode(pb1, INPUT_PULLUP);

```

```

pinMode(pb2, INPUT_PULLUP);
pinMode(rfLevelPin, INPUT);

delay(2000);
tft.fillScreen(backgroundColor);          // BG Color
change = true;
}      // end of setup

// LOOP
//-----

void loop() {

// the following is performed every seconds
//-----
static unsigned int tempo = 0;
static unsigned int counter = 0;
static byte tempoCounter = 0;
if( millis() < tempo + 500 ) return;
tempo = millis();

// check station change
if(!digitalRead(pb1)) {ChangeRadio(LOW); delay(500);}
else if(!digitalRead(pb2)) {ChangeRadio(HIGH); delay(500);}

// display rfLevel
int rfLevel = analogRead(rfLevelPin);
String legende = "Niv. Ant.: " + String(rfLevel);
tft.fillRect(130, 30, 200, 20, backgroundColor); // full screen resolution 135 x 240 pixels.
tft.setCursor(5, 30);
tft.setTextSize(2);
tft.print(legende);

// draw radio name on tft
if(change || (radioName != memoRadioName) || (radioTitle != memoRadioTitle)) {
    memoRadioName = radioName;
    memoRadioTitle = radioTitle;
    if( radioName == "" ) radioName = stations[radioNum];
    int startPos = radioName.lastIndexOf("/");
    int endPos = radioName.lastIndexOf(".");
    if(startPos > 0) radioName = radioName.substring(startPos + 1);
    if(endPos > 0) radioName = radioName.substring(0, endPos);

    int total = radioTitle.length();
    if(total > maxTitleLenght) {
        String concacRadioTitle = "";
        byte i = 0;
        char c1 = radioTitle.charAt(i);
        if(c1 == ' ') i++;
        while(i < total) {
            char c1 = radioTitle.charAt(i);
            char c2 = radioTitle.charAt(i+1);
            if((c1 == ' ') && (c2 == ' ')) i++;
            else concacRadioTitle += String(c1);
            i++;
        }
        radioTitle = concacRadioTitle.substring(0, maxTitleLenght);
        endPos = radioTitle.lastIndexOf(" ");
        if(endPos > 0) radioTitle = radioTitle.substring(0, endPos);
    }
    DrawDisplay();
}

// radio autostop
if( ++tempoCounter > (radioPlayingDelay * 60)) {
    tft.fillScreen(backgroundColor);
    tft.drawString("Allez, dodo", tft.width()/2, tft.height()/2);
    delay(2000);
    esp_deep_sleep_start();
}

}      // end of Loop

//
// LOOP2 the loop2 function also runs forever but as a parallel task
//-----

void loop2(void* pvParameters) {

    delay(2000);
    audio.forceMono(true);
    audio.setVolume(volume);          // Set vol 0...21 or volumeSteps

// here start the Core 0 loop
//-----

```

```

while(true) {
    audio.loop();
    vTaskDelay(1);    // delay(1);

    if(change) {
        change = false;
        ListenRadio();
        memoRadioName = radioName;
        radioName = "";
        radioTitle = "";
    }
} // end of while
} // end of loop2

//=====
// list of functions
//=====
//
// rfTransmitter : set Carrier (PWM aka LEDC)
//
void RfTransmitter() {
    ledcSetup(PWM_channel, rfFreq *1000, PWM_Res); // set frequency & resolution
    ledcAttachPin(rfOut, PWM_channel); // ASSIGN CHANNEL 0 TO GPIO-PIN
    ledcwrite(PWM_channel, duty);
}

//
// DrawDisplay() = function to draw the radiostation name
//
void DrawDisplay() {
    tft.setRotation(screenRotation);
    tft.fillScreen(backgroundColor);
    tft.setTextDatum(MC_DATUM);

    tft.setTextSize(2);
    String legende = "eRadio / PO " + String(rfFreq) + "kHz";
    tft.drawString(legende, tft.width()/2, tft.height()/2 -50);
    tft.drawString(radioName, tft.width()/2, tft.height()/2 +10 );
    tft.setTextSize(1);
    tft.drawString(radioTitle, tft.width()/2, tft.height()/2 +30 );
    legende = "Param:" + dnsName;
    tft.drawString(legende, tft.width()/2, tft.height()/2 +50);
}

// ListenRadio() functions to listen the web radio
//
void ListenRadio() {
    //Serial.print(F("ListenRadio() running in core ")); Serial.println(xPortGetCoreID());
    Serial.print(F("web radio station:")); Serial.println(stations[radioNum]);
    audio.connecttohost(stations[radioNum].c_str());
}

void audio_showstation(const char *info){
    Serial.printf("Station: %s\n", info);
    radioName = info;
}

void audio_showstreamtitle(const char *info) {
    Serial.print(F("showstreamtitle=")); Serial.println(info);
    radioTitle = info;
}

void printTitle(const char* info) {
    Serial.print(F("print_title=")); Serial.println(info);
}

void printInfo(const char* info) {
    Serial.print(F("print_info=")); Serial.println(info);
}

void audio_info(const char *info) {
    Serial.print(F("audio_info=")); Serial.println(info);
}

void audio_id3data(const char *info) {
    Serial.print(F("audio_id3data=" )); Serial.println(info);
}

void audio_bitrate(const char *info) {
    Serial.print(F("bitrate=" )); Serial.println(info);
}

```

```

void audio_commercial(const char *info) { //duration in sec
    Serial.print(F("commercial= ")); Serial.println(info);
}

void audio_icyurl(const char *info) { //homepage
    Serial.print(F("audio_icyurl= ")); Serial.println(info);
}

void audio_lasthost(const char *info) { //stream URL played
    Serial.print(F("lasthost= ")); Serial.println(info);
}

// ChangeRadio() change radiostation to listen
//_____

void ChangeRadio(bool s) {
    if(s) { if(++radioNum >= radioList) radioNum = 0; }
    else { if(--radioNum < 0) radioNum = radioList -1; }
    pref.putUChar("radio", radioNum);
    change = true;
}

// UpdateRadioList() store radio list to SPIFFS
//_____

void UpdateRadioList() {
    File f = SPIFFS.open("/stations.txt", FILE_WRITE);
    if(!f) Serial.println(F("Cannot open SPIFFS"));
    else {
        Serial.printf("list of the %d web radio:\n", radioList);
        for( byte i=0; i< radioList; i++ ) {
            Serial.printf("%d = ", i+1); Serial.println(stations[i]);
            f.println(stations[i]);
        }
        f.close();
    }
}

//
// sd card functions
//_____

void ListDir(fs::FS &fs, const char *dirname, uint8_t levels) {
    Serial.printf("Listing directory: %s\n", dirname);

    File root = fs.open(dirname);
    if(!root) {
        Serial.println(F("Error to open directory"));
        return;
    }
    if(!root.isDirectory()) {
        Serial.println(F("Not a directory"));
        return;
    }

    File file = root.openNextFile();
    while(file) {
        if(file.isDirectory()) {
            Serial.print(F("\tDIR: "));
            Serial.println(file.name());
            if(levels >0) ListDir(fs, file.name(), levels -1);
        }
        else {
            Serial.print(F("\tFILE: "));
            Serial.print(file.name());
            Serial.print(F("\tSIZE: "));
            Serial.println(file.size());
        }
        file = root.openNextFile();
    }
}

// end of ListDir()

void CopyFile(fs::FS &fs1, const char * path1, fs::FS &fs2, const char * path2) {
    Serial.printf("copying: %s/%s to %s/%s\n", fs1, path1, fs2, path2);
    File file1 = fs1.open(path1, FILE_READ);
    if( fs2.open(path2, FILE_READ)) Serial.printf("%s yet exists, delete it first\n", path2);
    File file2 = fs2.open(path2, FILE_WRITE);

    size_t n;
    uint8_t buf[64];
    while((n = file1.read(buf, sizeof(buf))) > 0) {
        Serial.println(n);
        file2.write(buf, n);
    }
}

```

```

    }
    file2.close();
}

void DeleteFile(fs::FS &fs, const char * path) {
    Serial.printf("Deleting file: %s/%s\n", fs, path);
    if (fs.remove(path)) Serial.println(F("File deleted"));
    else Serial.println(F("Delete failed"));
}

//
// HTML pages with buttons & inputs management
//
// actions done from the named_html[] as long as there are %PLACEHOLDER% keys.
// These palceholders will then be replaced with the actual HTML text to build the page once
// loaded.
// Replaces placeholder with button section in your web page
String processor(const String& var) {

    String h = "";

    // main webpage
    if(var == "RADIOPLACEHOLDER") {
        h = "<p><h3>station de eRadio: " + String(stations[radioNum]) + "</h3><p>";
        h += "<p><a href='/stationprec'><button class='button button3'></button></a>";
        h += "<a href='/stationsuiv'><button class='button button3'></button></a></p>";
        h += "<HR size=2 align=center>";

        h += "<p><h3>supprimer la station de eRadio: " + String(radioNum +1) + "</h3><p>";
        h += "<p><a href='/confirmdelete'><button class='button'>Supprimer
station</button></a></p>";

        h += "<p><h3>ajouter une station, saisir l'url:</h3><p>";
        h += "<form action='/addradio'><input type='text' name='r'>";
        h += "<INPUT class='button button3' type='submit' value='valider'></form></p>";

        h += "<HR size=2 align=center>";
        h += "<p><h3>Radio fr&eacute;quence: " + String(rfFreq) + "kHz</h3><p>";
        h += "<p><a href='/freqmoins'><button class='button button3'>-</button></a>";
        h += "<a href='/freqplus'><button class='button button3'>+</button></a></p>";

        h += "<p><h3>Taux de modulation: " + String(map(volume, 0, 21, 0, 100)) + "%</h3><p>";
        h += "<p><a href='/modulmoins'><button class='button button3'>-</button></a>";
        h += "<a href='/modulplus'><button class='button button3'>+</button></a></p>";

        h += "<HR size=2 align=center>";
        h += "<p><h3>rotation &eacute;cran:</h3><p>";
        h += "<p><a href='/rotation'><button class='button button3'>@</button></a></p>";

        h += "<p><h3>" + VERSION + "</h3></p></BODY></center></html>";
        return h;
    }

    // confirm delete radio station
    if(var == "CONFIRMDELETEPLACEHOLDER") {
        h = "<p><h3>confirmer la suppression de la station de eRadio: " + radioName + "</h3><p>";
        h += "<p><a href='/deleteradio'><button class='button'>Oui</button></a>";
        h += "<a href='/radio'><button class='button button2'>Non</button></a></p>";
        return h;
    }

    return String();
}
// end of processor()

```


Improved MW-AM transmitter

Description: the choose of a C-class amplifier

Now we can have an AM transmitter in MW range, the objective is to get some power so that the transmitter can be listen for more or less 10 meters inside a house. The power stage must be as low power as possible, so it can be forgotten powered-on as is, ready for any radio set to be able to display the audio of a the web radio station.

So no A-class amplifier: large consumption and high heating, a large radiator would be needed.

E-class would be great, but the modulated signal is not PWD.

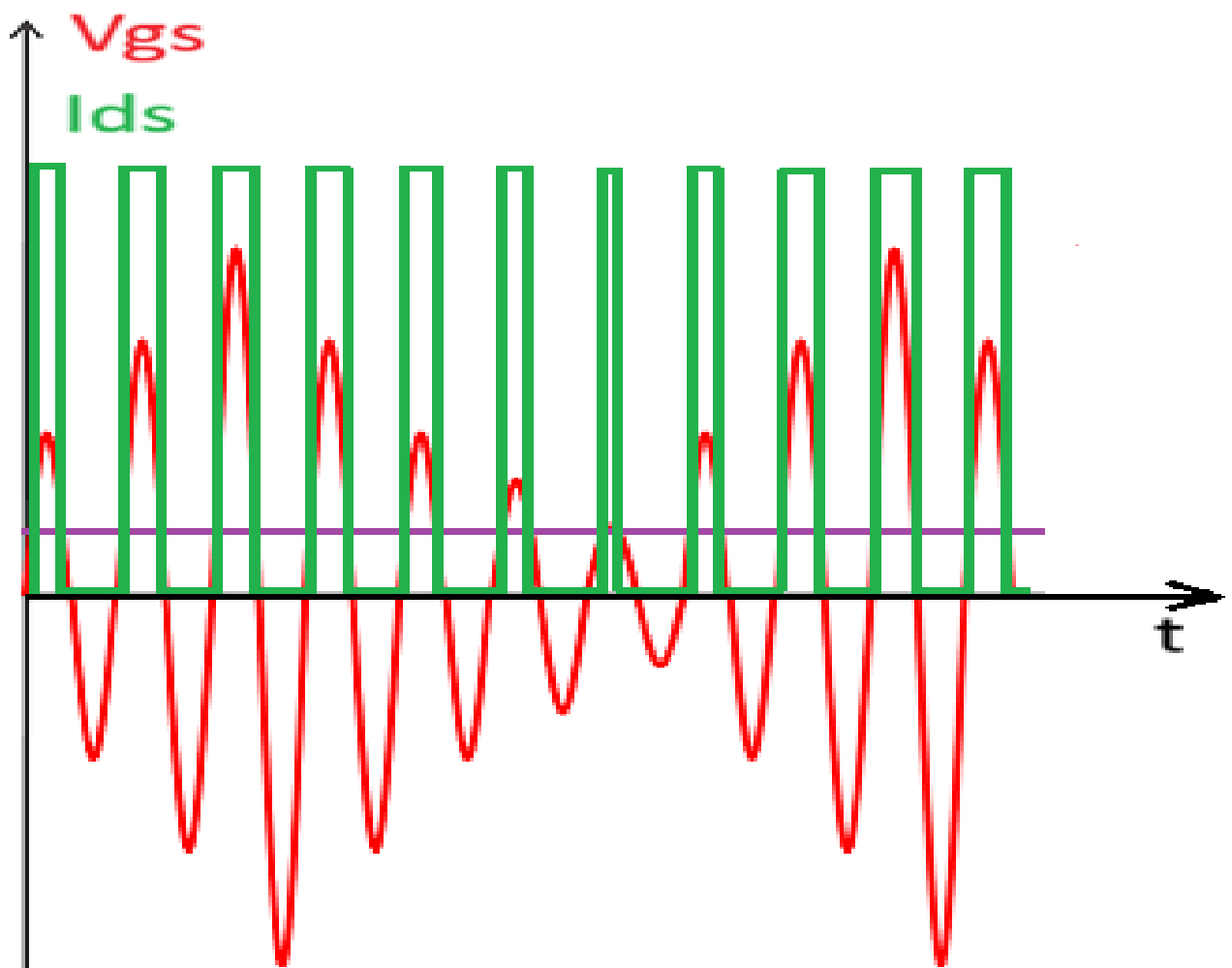
For curious guys here a very good explanation for beginners about E-class vs C-class RF amplifiers :

https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=http://www.norcalqrp.org/files/Class_E_Amplifiers.pdf&ved=2ahUKEwj10rfSrzWSAxUvNPdHeEBG4UQFnoECBgQAQ&usg=AOvVaw0lQb74_fU4M1KhcaFZcNsE

So, a C-class could be the best idea. This choice became definitive once I read transmitters projects from Marc Doigny and in particular its description here (in French sorry):

<http://server.idemdito.org/electro/histo/audio/amt2.htm>

Yes! It is possible to amplify an analog signal with a C-class amplifier. Here are explanations how it works:



Low input capacitor MOSFETs like 2N7000, BS170 or VN66AF have a threshold voltage around +2V.

Any V_{gs} voltage under this threshold voltage will not generate any I_d current. Instead, for any V_{gs} value above this threshold voltage will generate a I_d current under the formula

$$I_d = V_{cc} / Z_d$$

Where V_{cc} is the power voltage and Z_d the Drain load.

The draw above illustrates how change the I_d current according to the V_{gs} voltage: the time the MOSFET is like a switch-on interrupter is varying according to the amplitude of V_{gs} above the threshold voltage. More the V_{gs} is high voltage, more the time the V_{gs} sinusoidal signal will be above the threshold voltage, more the time the MOSFET is switch on is long.

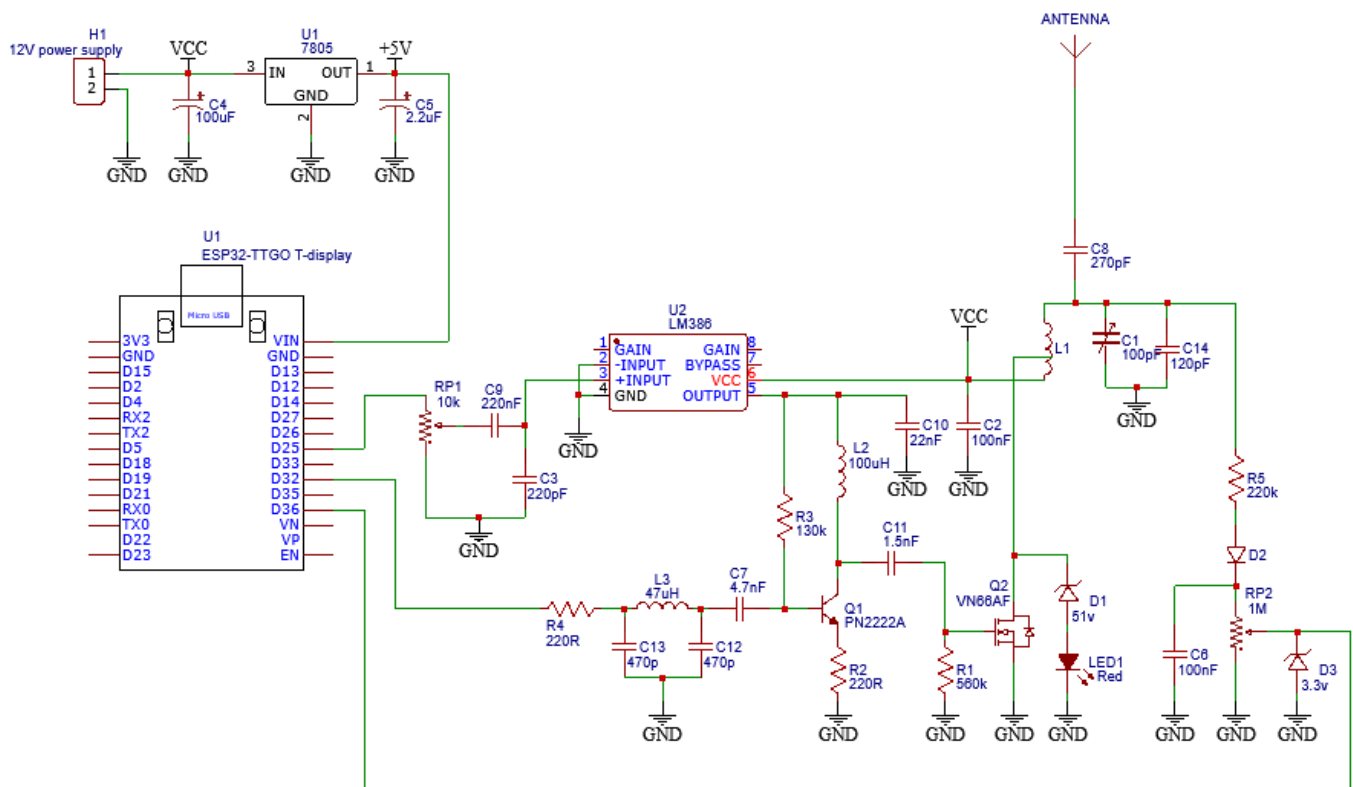
In fact, this stage simply looks like a PWM generator.

As the times the MOSFET is conducting is less than half a period, it is well a C-class amplifier. With no signal at V_{gs} there is no I_d , so no power consumption.

The most important thing to consider is this PWM generator is possible only if the carrier signal is NOT a square. The signal is above all not square, but with a marked slope like a sinusoidal or triangular signal can be.

So the square carrier signal available from the ESP32 output must be filtered.

The diagram of the ESP32 with a C-class transmitter



C12 + C13 + L3 is a low pass Butterworlf filter, the Q2 Base voltage is now the sinusoidal carrier signal.

R2 limits the Q2 gain, to lower distortion.

L2 is a tuned circuit with... the MOSFET input capacitor. VN66AF is about 50pF. VN66AF is hard to find now. It can be replaced by a pair of 2N7000 or BS170, in parallel connection. As these MOSFET, compared to a VN66AF, are half powerful and half input capacitor, it will be the same result replacing the VN66AF with 2 parallel connected 2N700 or BS170.

A IRF510 can also be used, however it has a 100pF input capacitor. So L2 may be adjusted to 47uH instead of 100uH. As it is a much more powerful MOSFET, may be the LM386 should be replace by a TDA2003 that allows a 18V power supply. I have not tested this configuration, as I do not own any IRF510 at the moment I write this paper.

L1 is coming from a ferrite frame tuning coil of an old radio. The small side is connected to power supply and the Drain, le large side to the antenna: this coil is like a step-up transformer. The voltage at the antenna can rise 200V!

This load is a tuned circuit. Depending of its quality factor, the Vds voltage can easily exceed the maximum MOSFET voltage, 60V, so a security must take place to limit the max Vds.

D1 is a 51V Zener, I replace it with 2* 27V Zener wired in series. It limits the Vds under the 60V maximum admmissive voltage for the MOSFET. When this security is active the LED1 is lighting.

As the antenna is seen as a capacitor, the C8 value must be respected. C1 is tuned to get the max voltage.

There is a way to have a look to the RF transmitted voltage level: R5 pickups the RF signal, D2 decodes it, D3 limits under the 3.3V allowed by the ESP32 inputs.

Vcc can be 9V, enough the transmitter broadcasts throughout the house

List of materials

1. 1x ESP32 TTGO T-display or any ESP32 models with their own DAC
2. 1x 7805 5V voltage regulator for the ESP32 power supply
3. 1x LM386
4. 1x PN2222A or 2N2222A (the A is important for better performances)
5. 1x VN66AF or 2x in parallel wired 2N7000 or BS170
6. 1x 100uH coil,
7. 1x ferrite frame tuning coil of an old radio
8. 1x 100pF adjustable capacitor
9. Few resistors, capacitors, zeners....